



Heaps.

Heap is the first data structure exhibiting a non-trivial algorithmic mechanism. It allows extracting the min (max) value at a logarithmic time cost. It plays a crucial role in finding shortest-path algorithms, which we will see at the end of the course. In this recitation we will:

1. Outline heaps in brief.
2. Practice correction proving by proving the correction of heap-sort and heapify-down.
3. Present median-heap, an application of heap to maintaining dynamically the median value.

1 Review

Definition 1. Consider the sequence $H = H_1, H_2, \dots, H_n \in \mathbb{R}$. We will say that H is a min-Heap if for every $i \in [n]$, we have that $H_i \leq H_{2i}, H_{2i+1}$ ¹.

Max-Heap is defined the same way, but by flipping the directions of the inequalities.

That definition is equivalent to the following recursive definition: Consider an almost complete binary tree where each node is associated with a number. Then, we will say that this binary tree is a heap if the root's value is lower than its children's values, and each subtree defined by its children is also a heap. There is a one-to-one mapping between these definitions by setting the array elements on the tree in order.

Checking vital signs. Are the following sequences are heaps?

1. 1,2,3,4,5,6,7,8,9,10 (Y)
2. 1,1,1,1,1,1,1,1,1,1 (Y)
3. 1,4,3,2,7,8,9,10 (N)
4. 1,4,2,5,6,3 (Y)

¹When we think of the values at indices greater than n as $H_{i>n} = -\infty$

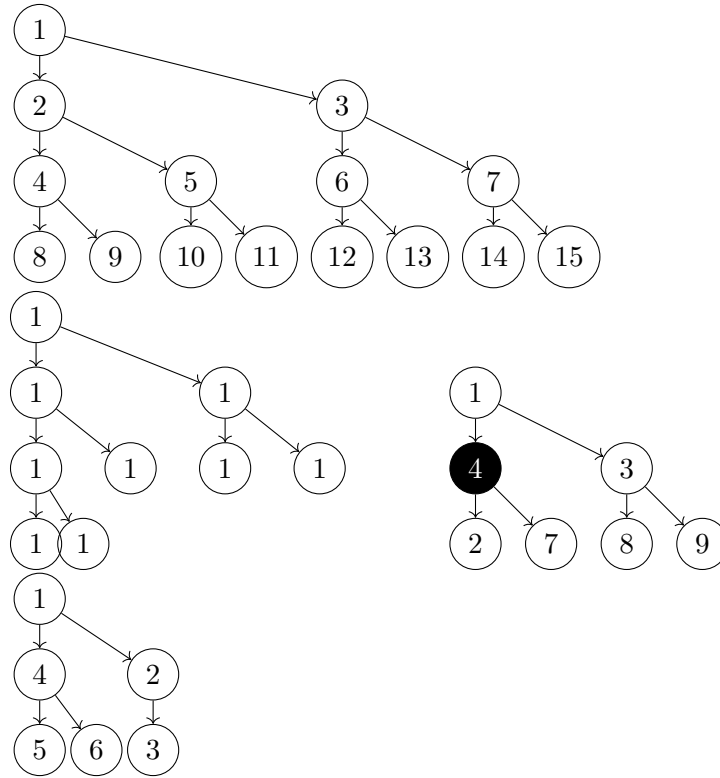


Figure 5.1: The trees representations of the heaps above. The node which fails to satisfy the Heap inequality is colored.

Remark 1. Why do we² stick to the array representation of heaps when the binary tree representation might seem more convenient? The reason is that the array representation exhibits an in-place sorting algorithm, namely a sorting algorithm that does not allocate additional memory for the target input.

2 Heapsort

We will start by introducing the heap-sort algorithm and providing a proof of its correctness, assuming the correctness of the heapify-down. The heapify-down assumes it is over the root of two heaps, the root itself might not satisfy the heap inequality, and in that case, it is bubbled down until the array becomes a heap. The heap-sort is given in Algorithm 1.

Correctness. We are going to prove the following statement. For convenience, we use the following notation: $A_i^{(t)}$ is the i th item of A at iteration t immediately after line (3), and $A_i'^{(t)}$ is the i th item of A at iteration t after line (5) (or equivalently at the beginning of the $t + 1$ iteration). Observe that by the definition of the algorithms, line (3) touches only the 1st and the $(n - t - 1)$ th elements, so $A_j'^{(t-1)} = A_j^{(t)}$ for any $j \neq 1, n - t - 1$.

Claim 1. *At the end of the i th iteration:*

²Here: we = any entity that teaches data structure, not the HUJI course in particular.

```

1  $A \leftarrow \text{Build-Heap}(A)$ 
2 for  $i \in [n]$  do
3    $\text{swap } A_1 \leftrightarrow A_{n-i+1}$ 
4    $\text{heapsize}(A) \leftarrow n - i$ 
5    $\text{heapify}(A, 1)$ 
6 end
7 return  $A$ 

```

Algorithm 1: Heap-sort(A)

1. $A_{n-i+1}^{(i)}, A_{n-i+2}^{(i)}, \dots, A_n^{(i)}$ are the i largest elements of A placed in order
2. $A_1^{(i)}, A_2^{(i)}, \dots, A_{n-i}^{(i)}$ is a maximum heap.

Proof. By induction.

1. Base.

- $A_n^{(1)}$ is set in line (3) to be the root of the heap and therefore is the maximum of A . Since, at line (4), we decrease the length size of the heap, and by the correctness of heapify, we have that $A_n^{(1)}$ is untouched when executing heapify-down, or in other words, $A_n^{(1)} = A_n^{(1)} = A_1^{(0)} = \max A$, so we have the first part of the claim.
- Since earlier in line (5), any other element but $A_1^{(0)}, A_n^{(0)}$ was untapped, we have that for any $j \in [2, n-1]$, $A_j^{(1)} = A_j^{(0)}$. Thus, for any $j \in [2, n-1]$, A_j satisfies the heap inequality, or in other words, $A_1^{(1)}$ is a root of two heaps.

Hence, by the correctness of the heapify-down algorithm and the decreasing of the heap length at line (4), we have that $A_1^{(1)}, A_2^{(1)}, \dots, A_{n-1}^{(1)}$ is a heap, and we have the correctness of the second part of the claim.

2. Assumption. Assume the correctness of the claim for any $i' < i$.

3. Step. Consider the i th iteration.

- By the first part of the induction assumption, $A_1^{(i-1)}$ is a root of the heap $A_1^{(i-1)}, A_2^{(i-1)}, \dots, A_{n-i+1}^{(i-1)}$ and therefore is their maximum.

So after the swapping in line (3), we get that $A_{n-i+1}^{(i)}$ is the element which is greater than $n-i$ elements in A .

By using the second part of the induction assumption, we know that it is also less than $A_{n-i+2}^{(i-1)}, A_{n-i+3}^{(i-1)}, \dots, A_n^{(i-1)}$, so after line (3) and by that $A_{n-i+2}^{(i-1)}, A_{n-i+3}^{(i-1)}, \dots, A_n^{(i-1)}$ are being the $i-1$ largest elements placed in order, we have that:

$$A_{n-i+1}^{(i)}, A_{n-i+2}^{(i)}, A_{n-i+3}^{(i)}, \dots, A_n^{(i)} = A_{n-i+1}^{(i)}, A_{n-i+2}^{(i-1)}, A_{n-i+3}^{(i-1)}, \dots, A_n^{(i-1)}$$

are the i largest elements placed in order.

- Similarly to the base case, line (4) confines the 'heap' into $A_1^{(i)}, \dots, A_{n-i}^{(i)}$. Any element $A_j^{(i-1)}$, such that $j \neq 1, n-i+1$, is untouched by line (3), and therefore $A_j^{(i)} = A_j^{(i-1)}$ for any $j \in [2, n-i]$. Thus, all the elements in the range satisfy the heap inequality, and by the correctness of heapify-down, we get that $A_1^{(i)}, \dots, A_{n-i}^{(i)}$ is a heap.

□

3 Heapify-Down

We are now ready to prove correction of the bubbling routine.

input: Array $a_1, a_2, \dots, a_n$

```

1 next  $\leftarrow i$ 
2 left  $\leftarrow 2i$ 
3 right  $\leftarrow 2i + 1$ 
4 if left  $< n$  and  $H_{\text{left}} < H_{\text{next}}$  then
5   | next  $\leftarrow$  left
6 end
7 if right  $< n$  and  $H_{\text{right}} < H_{\text{next}}$  then
8   | next  $\leftarrow$  right
9 end
10 if  $i \neq \text{next}$  then
11   |  $H_i \leftrightarrow H_{\text{next}}$ 
12   | Heapify-down(next)
13 end
```

Algorithm 2: Heapify-down

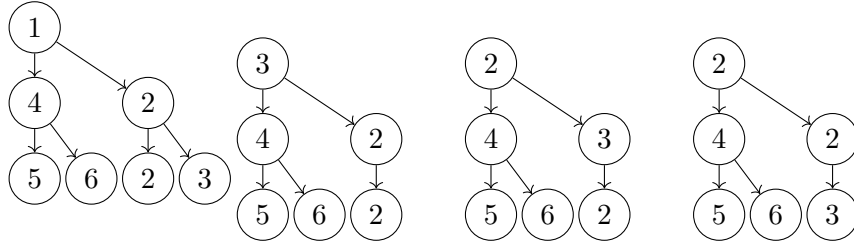


Figure 5.2: Running Example: Extracting the minimum value of the heap is done by first removing the root of the heap and then maintaining the heap property by moving the rightmost leaf to the root of the heap and then bubbling it down through the heapify method.

Claim 2. Assume that H satisfies the Heap inequality for all the elements except the root. Namely for any $i \neq 1$ we have that $H_i \leq H_{2i}, H_{2i+1}$. Then applying Heapify-down on H at index 1 returns a heap.

Proof. By induction on the heap size.

- Base, Consider a heap at the size at most three and prove for each by considering each case separately. (lefts as exercise).

- Assumption, assume the correctness of the Claim for any tree that satisfies the heap inequality except the root, at size $n' < n$.
- Induction step. Consider a tree at size n and assume w.l.g (why could we?) that the right child of the root is the minimum between the triple. Then, by the definition of the algorithm, at line 9, the root exchanges its value with its right child.

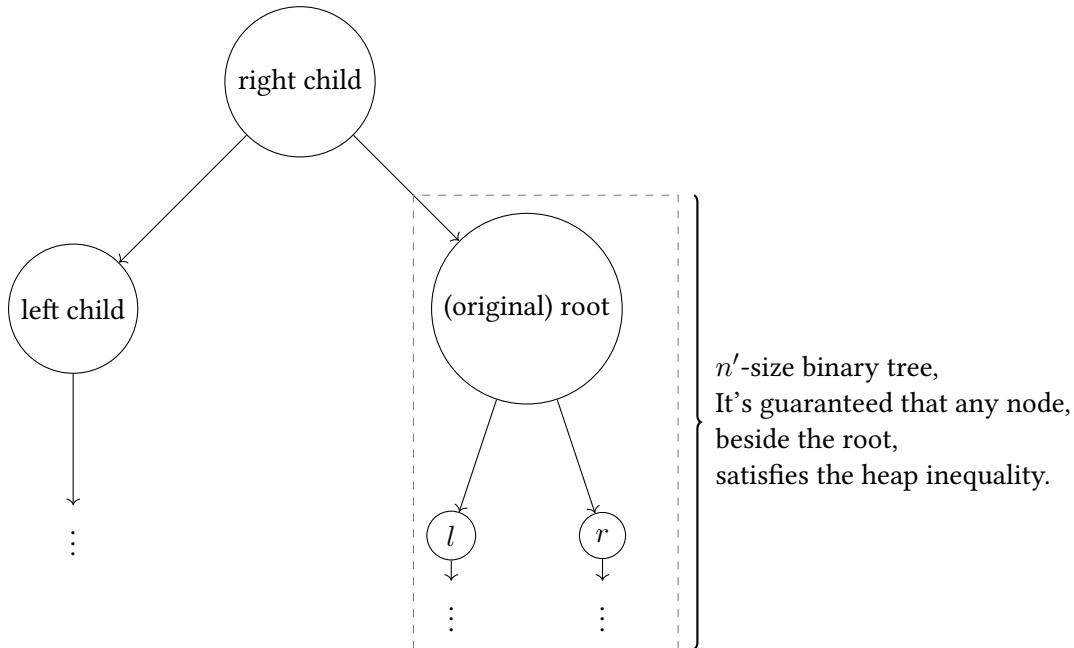


Figure 5.3: The heap after the exchange, at line (9). The induction assumption is applied over the right subtree, which is framed in gray.

Given that before the swapping, all the elements of the heap, except the root, had satisfied the heap inequality, we have that after the exchange, all the right subtree's elements, except the root of that subtree (which is the original root), still satisfy the inequality.

As the size of the right subtree is at most $n-1 < n$, we can use the induction assumption and have that after calling `heapify-down` at line (12), the right subtree is a heap.

Now, as the left subtree remains the same (the values of the nodes on the left side weren't touched), we have that this subtree is also a heap.

So to complete, namely to show that all the elements satisfy the heap inequality, it's left to show that the new tree's root is smaller than its children.

Suppose that is not the case, then it's clear that the root of the right subtree (heap) is smaller than the new root. Combining the fact that its origin must be the right subtree, we have a contradiction to the fact that the original right subtree was a heap (as its root wasn't the minimum element in that subtree).

□

4 Example, Median Heap

Task: Write a datastructure that support insertion and deletion at $O(\log n)$ time and in addition enable to extract the median in $O(\log n)$ time.

Solution. We will define two separate Heaps, the first will be a maximum heap and store the first $\lfloor n/2 \rfloor$ smallest elements, and the second will be a minimum heap and contain the $\lceil n/2 \rceil$ greatest elements. So, it's clear that the root of the maximum heap is the median of the elements. Therefore to guarantee correctness, we should maintain the balance between the heap's size. Namely, promising that after each insertion or extraction, the difference between the heap's size is either 0 or 1.

```

input: Array  $H_1, H_2, \dots, H_n, v$ 
1 if  $H_{\max,1} \leq v \leq H_{\min,1}$  then
2   if  $\text{size}(H_{\max}) - \text{size}(H_{\min}) = 1$  then
3     Insert-key (  $H_{\min}, v$  )
4   end
5   else
6     Insert-key (  $H_{\max}, v$  )
7   end
8 end
9 else
10  median  $\leftarrow$  Median-Extract  $H$ 
11  if  $v < \text{median}$  then
12    Insert-key (  $H_{\max}, v$  )
13    Insert-key (  $H_{\min}, \text{median}$  )
14  end
15  else
16    Insert-key (  $H_{\min}, v$  )
17    Insert-key (  $H_{\max}, \text{median}$  )
18  end
19 end

```

Algorithm 3: Median Insert key.

```

input: Array  $H_1, H_2, \dots, H_n$ 
1 median  $\leftarrow$  extract  $H_{\max}$ 
2 if  $\text{size}(H_{\min}) - \text{size}(H_{\max}) > 0$  then
3   temp  $\leftarrow$  extract  $H_{\min}$ 
4   Insert-key (  $H_{\max}, \text{temp}$  )
5 end
6 return median

```

Algorithm 4: Median-Extract.

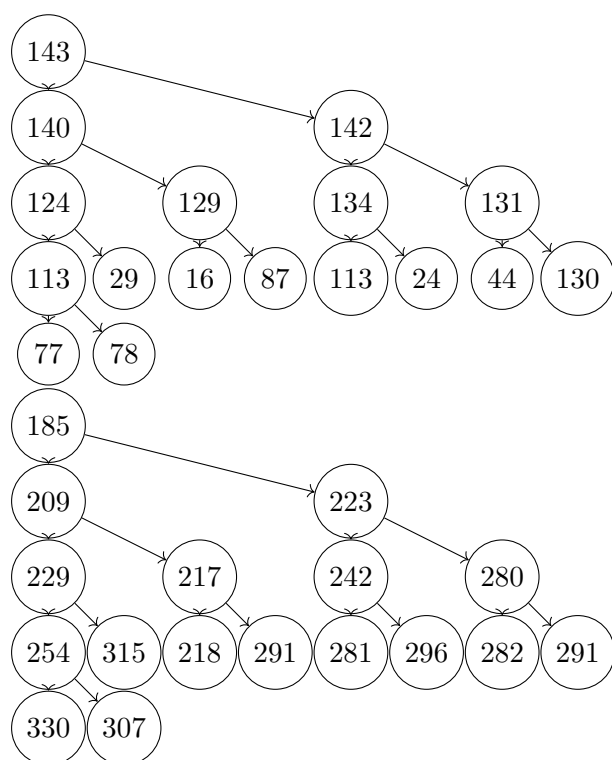


Figure 5.4: Example for Median-Heap, the left and right trees are maximum and minimum heaps.